

# The solution:

# Model Context Protocol

## (MCP)

## A universal adapter for tools

### What is MCP?

MCP (Model Context Protocol) is an **open standard** created by Anthropic and now hosted by The Linux Foundation. It allows AI agents to connect to external tool servers through a universal protocol. Think of it as USB for AI tools —any MCP-compatible server can provide tools to any MCP-compatible client, regardless of the AI framework being used.

### From ADK documentation:

“ADK helps you both use and consume MCP tools in your agents. Refer to the MCP Tools documentation for code samples and design patterns that help you use ADK together with MCP servers.”:

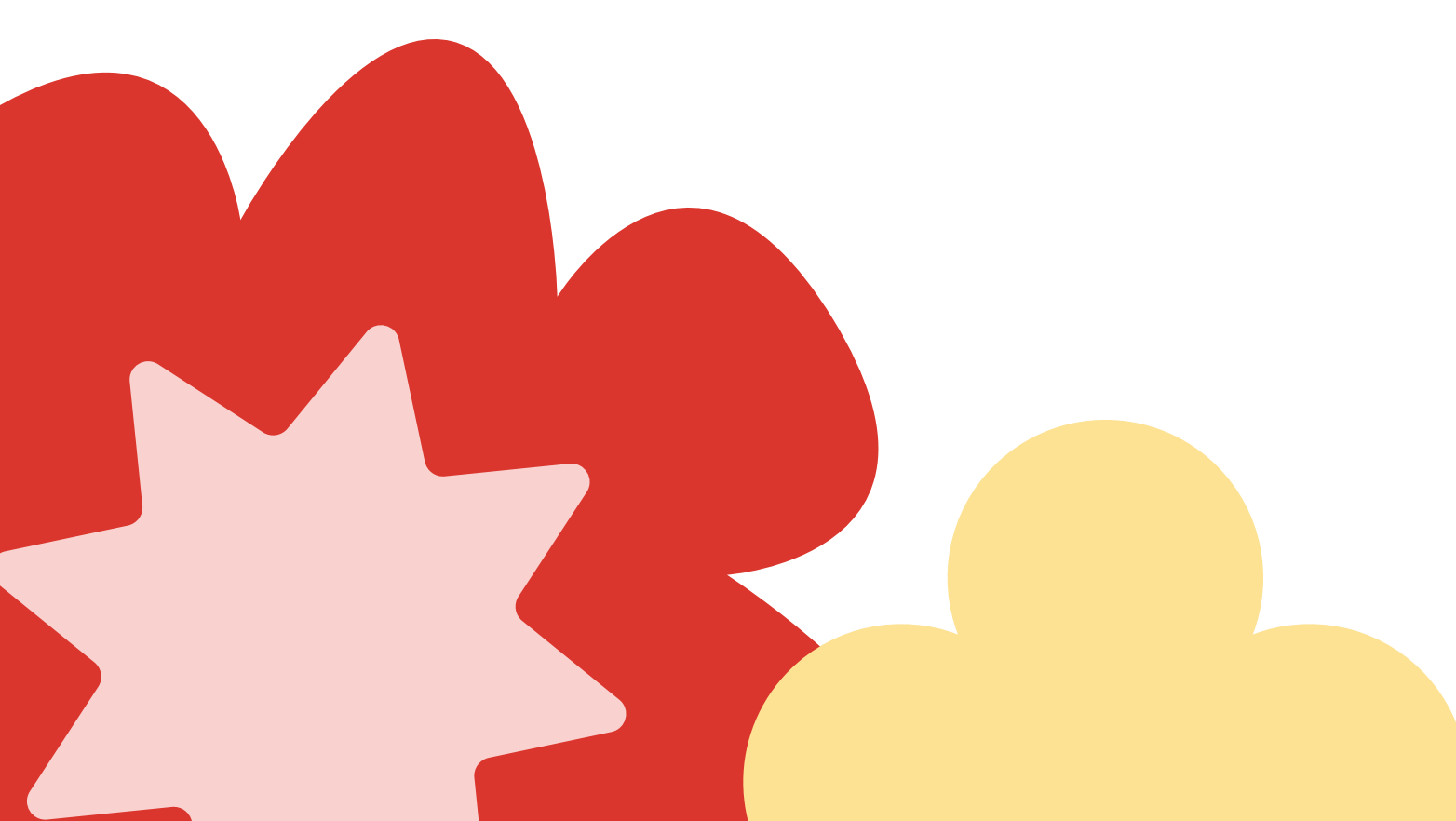
### Two integration patterns with ADK:

ADK supports two MCP integration patterns:

- 1 **Using Existing MCP Servers:** Your ADK agent acts as an MCP client, consuming tools from external servers (this module)
- 2 **Exposing ADK Tools via MCP:** You build an MCP server that wraps your ADK tools, making them accessible to any MCP client (advanced, see docs)

This module focuses on **Pattern 1: Connecting to pre-existing MCP servers to extend your agent's capabilities.**

Reference: [ADK docs - MCP tools](#) | [Official MCP documentation](#)



# Why MCP matters

## The power of standardization:

| Without MCP                                 | With MCP                                        |
|---------------------------------------------|-------------------------------------------------|
| Each tool needs custom integration code     | One integration pattern works for all tools     |
| Each tool needs custom integration code     | Tools work across Claude, GPT, Gemini, and more |
| You maintain every integration yourself     | Community and vendors maintain their tools      |
| Limited to what's built into your framework | Access to a growing ecosystem of 100s of tools  |

## Industry adoption:

- ✓ Anthropic created the protocol (November 2024)
- ✓ OpenAI officially adopted MCP (March 2025)
- ✓ Google ADK provides native MCP support via [McpToolset](#)
- ✓ The Linux Foundation now hosts MCP as an open standard

## What this means for you:

- **Write once, use anywhere:** MCP tools work across AI frameworks
- **Leverage the ecosystem:** Use tools built by specialists (databases, APIs, cloud services)
- **Stay current:** As new MCP servers are published, your agent can use them immediately
- **Separation of concerns:** Focus on your agent's logic, let tool maintainers handle integrations





# Finding MCP servers

Before building custom tools, check if an MCP server already exists for your use case.

## Official resources:

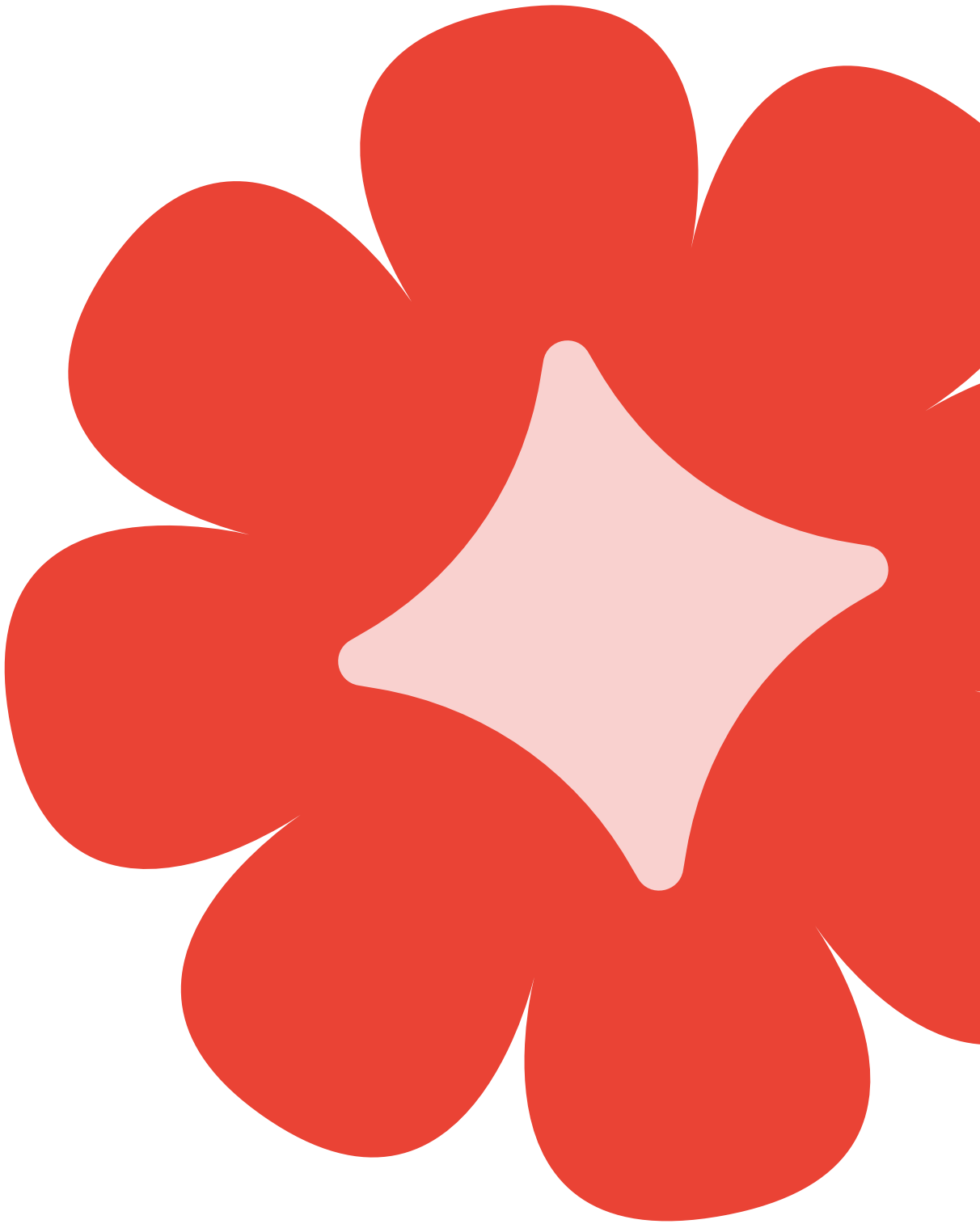
| Resource            | Description                                | Link                                                                                                  |
|---------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| MCP server registry | Official searchable catalog of MCP servers | <a href="https://registry.modelcontextprotocol.io">registry.modelcontextprotocol.io</a>               |
| Reference servers   | Official servers maintained by MCP team    | <a href="https://github.com/modelcontextprotocol/servers">github.com/modelcontextprotocol/servers</a> |
| MCP documentation   | Full protocol specification and guides     | <a href="https://modelcontextprotocol.io">modelcontextprotocol.io</a>                                 |

## Popular MCP servers include:

- **Filesystem:** Read/write files and directories
- **GitHub:** Repository management, issues, PRs
- **Slack:** Send messages, read channels
- **PostgreSQL/MySQL:** Database queries
- **Notion:** Document and database access
- **Google Drive:** File storage access

Tip:

Search the MCP Registry before writing custom tools—the capability you need may already exist.



# How MCP works in ADK

## Integration flow:

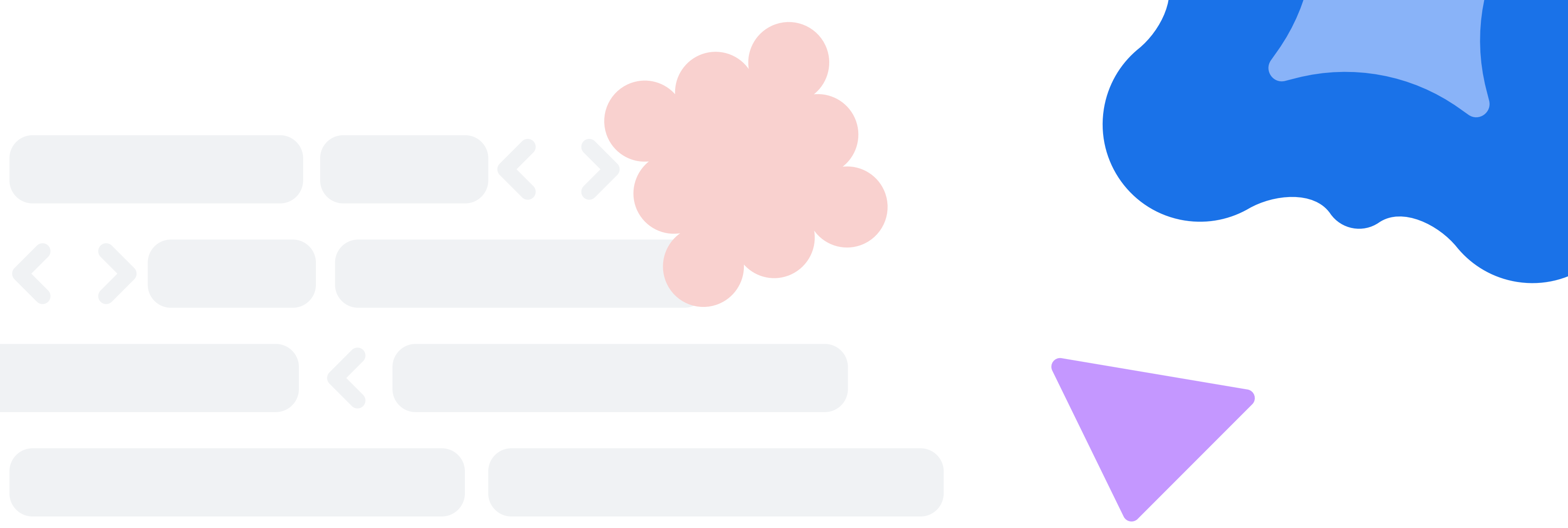
```
None
graph LR
  A[ADK Agent] --> B[McpToolset]
  B --> C[MCP Server]
  C --> D[Tools: list_directory, read_file, etc.]

  style A fill:#e1f5ff
  style B fill:#ffeb99
  style C fill:#eeffee
```

## The process:

- 1 MCP servers expose tools via the standardized MCP protocol
- 2 ADK agents connect to servers using **McpToolset**
- 3 Tools are discovered automatically, no manual registration needed
- 4 Your agent uses these tools exactly like any other ADK tool

ADK agents connect to MCP servers through **McpToolset**, automatically discovering available tools





# Core concepts

## 1. The `McpToolset` class

From ADK documentation:

“The `McpToolset` class can be directly added to your agent’s tools list, enabling seamless connection to an MCP server, discovery of its tools, and making them available for your agent to use.”

Reference: [ADK docs - MCP tools](#)

What `McpToolset` does:

- 1 Connects to an MCP server using connection parameters
- 2 Discovers available tools from the server automatically
- 3 Proxies tool calls from your agent to the MCP server
- 4 Returns results back to your agent

How to use:

Python

```
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

agent = LlmAgent(
    model='gemini-2.5-flash',
    name='my_agent',
    instruction="Use available tools to help users.",
    tools=[
        McpToolset(
            connection_params=StdioConnectionParams(
                server_parameters=StdioServerParameters(
                    command='npx',
                    args=['-y', '@some/mcp-server'],
                ),
            ),
        ],
    )
```

What to notice:

- `McpToolset` goes in the `tools` list like any other tool
- Connection parameters tell ADK how to reach the MCP server
- Tools from the server become available to your agent automatically



## 2. Connection types

ADK supports two ways to connect to MCP servers:

### `StdioConnectionParams` (local servers)

Launches an MCP server as a local subprocess on your machine.

```
Python
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

# Local server connection
connection = StdioConnectionParams(
    server_params=StdioServerParameters(
        command='npx', # Command to run
        args=['-y', '@modelcontextprotocol/server-filesystem', '/path'], # Arguments
    ),
)
```

**When to use:** Development, testing, single-user scenarios.

---

### `SseConnectionParams` (remote servers)

Connects to an MCP server running remotely via HTTP.

```
Python
from google.adk.tools.mcp_tool.mcp_session_manager import SseConnectionParams

# Remote server connection
connection = SseConnectionParams(
    url="https://your-mcp-server.example.com/sse",
    headers={'Authorization': 'Bearer YOUR_TOKEN'},
)
```

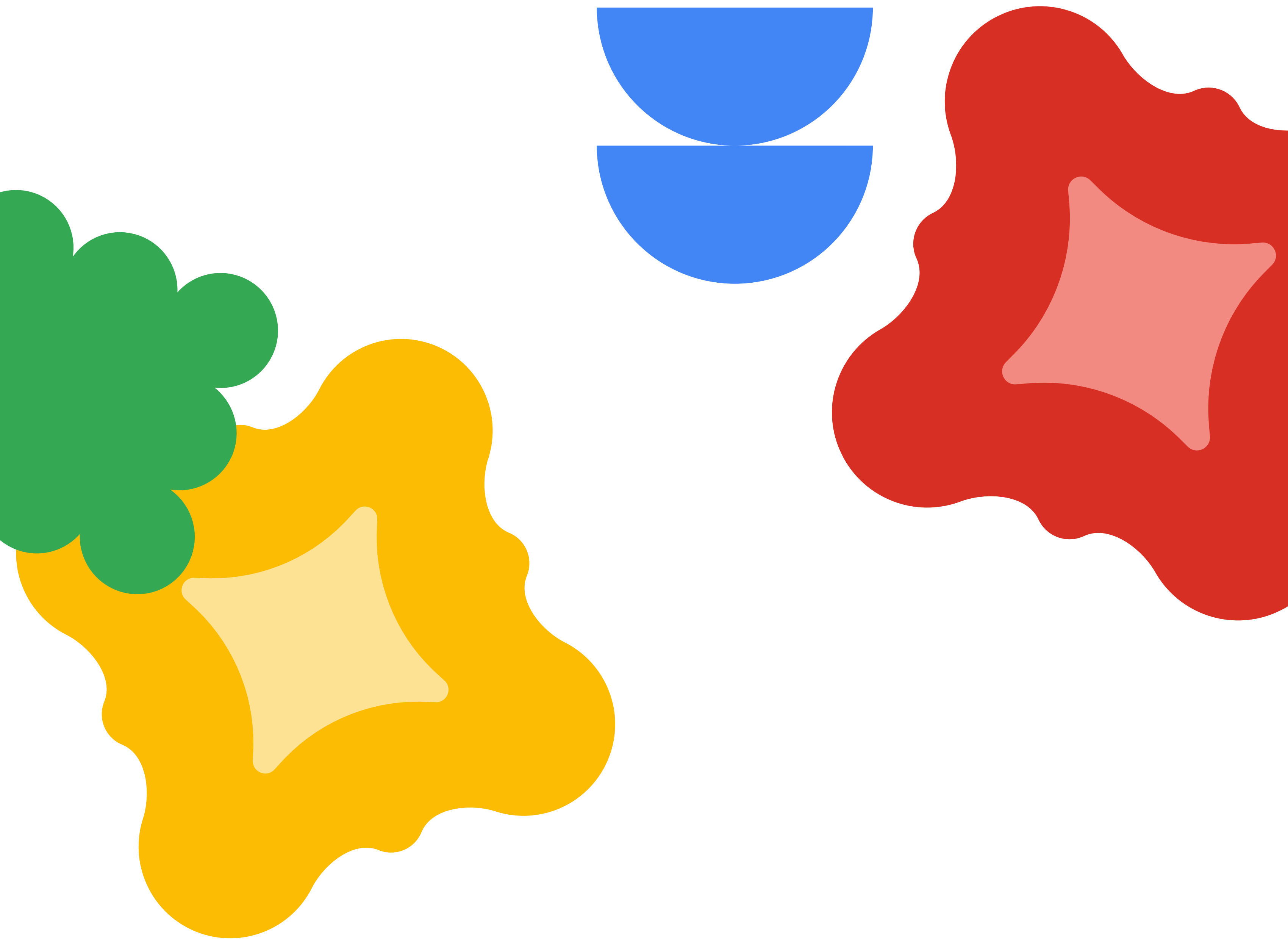
**When to use:** Production deployments, cloud-hosted MCP servers.

Reference: [ADK docs - MCP tools](#)



Connection types comparison:

| Aspect          | StdioConnectionParams | SseConnectionParams            |
|-----------------|-----------------------|--------------------------------|
| Server location | Local (subprocess)    | Remote (HTTP)                  |
| Setup           | Simple (npx command)  | Requires server infrastructure |
| Best for        | Development, testing  | Production, cloud              |
| Scalability     | Single user           | Multiple users                 |





### 3. Tool filtering

MCP servers may expose many tools, but you might only want your agent to use specific ones. The `tool_filter` parameter lets you control which tools are available.

#### Industry adoption:

- ✓ **Security:** Limit to read-only tools, exclude dangerous operations
- ✓ **Simplicity:** Give agent only what it needs for its task
- ✓ **Focus:** Reduce confusion from too many tool options

Reference: [ADK docs - MCP tools](#)

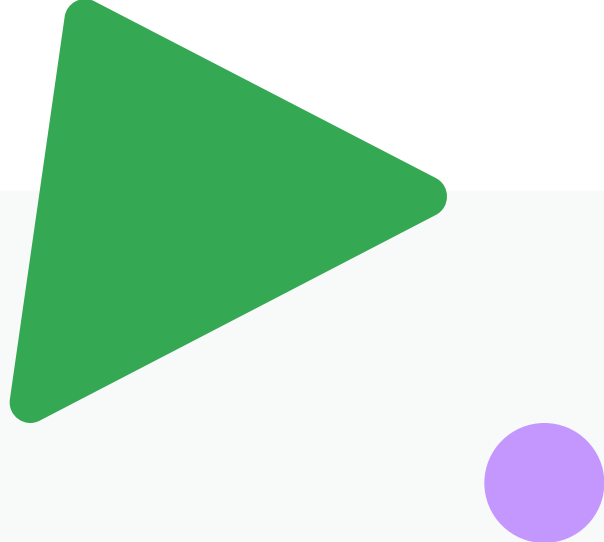
#### How to use:

```
Python
McpToolset(

    connection_params=StdioConnection
    Params(...),

    tool_filter=['list_directory',
                'read_file'], # Only these tools
)
```

#### Example: Restricting filesystem access



```
Python
# ✓ Good: Only expose safe, read-only tools
McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command='npx',
            args=['-y', '@modelcontextprotocol/server-filesystem', '/allowed/
path'],
        ),
    ),
    tool_filter=['list_directory', 'read_file'], # No write access
)

# ✗ Risky: Exposing all tools including write operations
McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command='npx',
            args=['-y', '@modelcontextprotocol/server-filesystem', '/'],
        ),
    ),
    # No filter = all tools exposed, including write_file, delete, etc.
)
```

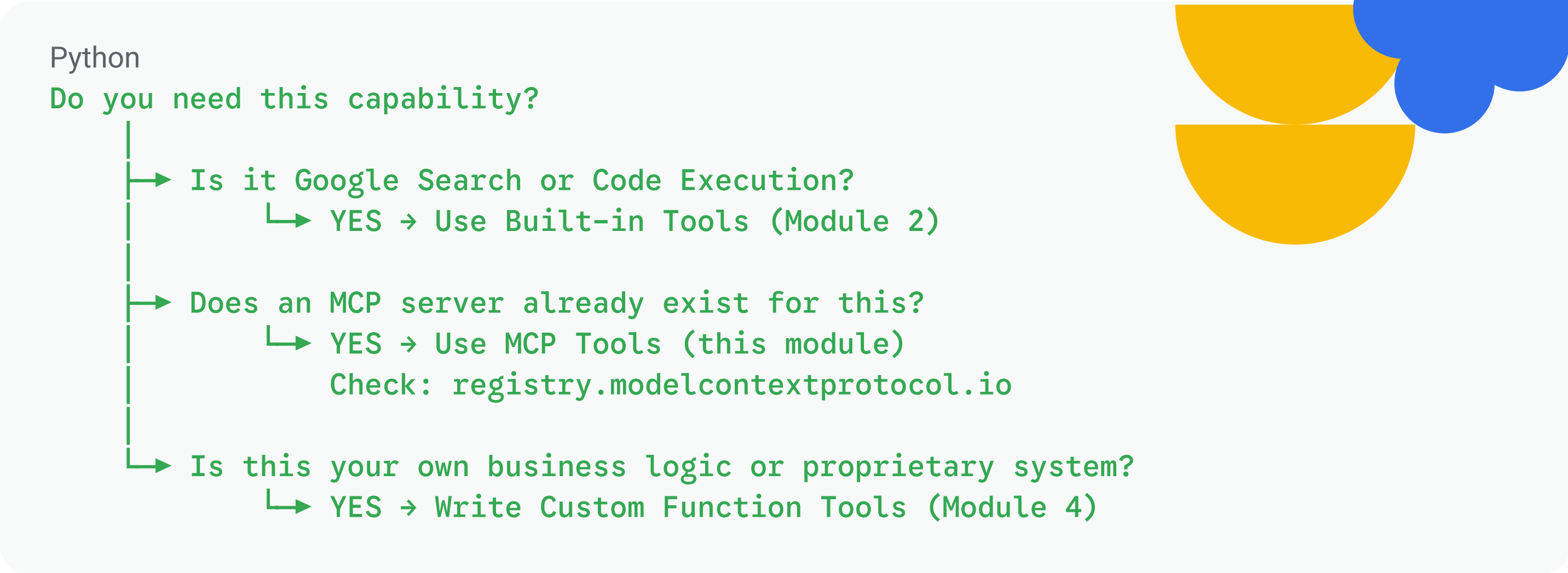
**Best practice:** Always use `tool_filter` in production to expose only the tools your agent needs.



## 4. Choosing the right tool type

When should you use MCP tools versus built-in tools or custom function tools?

The decision framework:



Key differences that matter:

| Aspect            | MCP tools                  | Built-in tools | Custom function tools |
|-------------------|----------------------------|----------------|-----------------------|
| Who maintains it  | Community/vendors          | ADK team       | You                   |
| Setup effort      | Connect and configure      | Import         | Write code            |
| Portability       | Works across AI frameworks | ADK only       | ADK only              |
| Customization     | Limited to server options  | Limited        | Full control          |
| Time to implement | Minutes                    | Minutes        | Hours to days         |

MCP tools are ideal when:

- ✓

The capability is **common** (files, databases, popular APIs)
- ✓

Someone else has **already solved** the integration problem
- ✓

You want **portability** across AI frameworks
- ✓

You prefer **maintained tools** over writing your own

Custom function tools are better when:

- ✓

The logic is **unique** to your business
- ✓

You need **full control** over behavior and error handling
- ✓

You're connecting to **proprietary** internal systems
- ✓

**No MCP server** exists for your specific use case

Real-world decision example:

| Need                         | Decision                                                     | Rationale                                         |
|------------------------------|--------------------------------------------------------------|---------------------------------------------------|
| Read files from disk         | MCP:<br><code>@modelcontextprotocol/server-filesystem</code> | Filesystem access is common; MCP server exists    |
| Query PostgreSQL database    | MCP:<br><code>@modelcontextprotocol/server-postgres</code>   | Database access is common; MCP server exists      |
| Calculate shipping cost      | Custom function tool                                         | Business-specific logic; only you know your rates |
| Access internal employee API | Custom function tool                                         | Proprietary system; no public MCP server          |

# Hands-on example

## Building a file reader assistant with MCP tools

What you'll build: An agent that can list and read files using the filesystem MCP server.  
Prerequisites: Node.js and npm installed (for the npx command)

### Step 1: Create the project

Shell

```
adk create geography_assistant
cd geography_assistant
```

What this does:

- Creates a new ADK project directory
- Sets up the basic agent structure
- Creates `agent.py` with default agent code





## Step 2: Create a test folder with files

Shell

```
mkdir -p my_files
echo "Hello, this is a test file." > my_files/hello.txt
echo "This is another file with different content." > my_files/notes.txt
```

What this does:

- Creates a folder that the MCP server will access
- Adds sample files for the agent to read



## Step 3: Write the agent

Replace the contents of `agent.py` with:

Python

```
"""
File Reader Assistant Agent
Demonstrates MCP tools integration with ADK using the filesystem MCP server.

Reference: https://google.github.io/adk-docs/tools-custom/mcp-tools/
"""

import os
from google.adk.agents import LlmAgent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters

# Define the folder to allow file access (must be absolute path)
ALLOWED_PATH = os.path.abspath("./my_files")

# Create the folder if it doesn't exist
os.makedirs(ALLOWED_PATH, exist_ok=True)

# Create the agent with MCP filesystem tools
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='file_reader_assistant',
    description='Helps users read and explore files using MCP tools.',
    instruction="""
    You are a file reader assistant that helps users explore files.
    """
```



Your capabilities:

- List files in directories using `list_directory`
- Read file contents using `read_file`

When helping users:

1. Use `list_directory` to show available files
2. Use `read_file` to display file contents when asked
3. Describe what you find in a helpful way

Always be clear about which folder you're working with.

```
"""  
tools=[  
    McpToolset(  
        connection_params=StdioConnectionParams(  
            server_params=StdioServerParameters(  
                command='npx',  
                args=[  
                    '-y',  
                    '@modelcontextprotocol/server-filesystem',  
                    ALLOWED_PATH,  
                ],  
            ),  
        ),  
        # Filter to only expose safe, read-only tools  
        tool_filter=['list_directory', 'read_file'],  
    ),  
],  
)
```

## Code walkthrough

### Lines 9–14: Setup and configuration

- Import `McpToolset` and connection classes from ADK
- Import `StdioServerParameters` from the MCP package
- Define `ALLOWED_PATH` as an absolute path to the files folder

### Lines 17–18: Create folder

- Ensure the target folder exists before the agent runs

### Lines 21–47: Create agent with MCP tools

- `McpToolset` connects to the filesystem MCP server
- `StdioConnectionParams` launches the server via `npx`
- `tool_filter` limits tools to `list_directory` and `read_file`
- Instruction guides the agent on how to use these tools



## Step 4: Run and test

```
Shell
adk web
```

Navigate to <http://localhost:8000> in your browser.

## Step 5: Test scenarios

### Test 1: List files

You: **What files are in the folder?**

#### Expected behavior:

- Agent calls **list\_directory** tool via MCP
- Tool returns list of files in **my\_files** folder
- Agent presents the file list to you

#### What to notice:

- Agent discovered and used the MCP tool automatically
- No custom tool code was needed—the MCP server provided everything

### Test 2: Read a file

You: **Show me the contents of hello.txt**

#### Expected behavior:

- Agent calls **read\_file** tool with the file path
- Tool returns the file contents
- Agent presents the contents to you

#### What to notice:

- Agent correctly invoked the MCP tool with the right argument
- File contents are returned through the MCP protocol

### Test 3: Multiple operations

You: **List the files and then read the notes.txt file**

#### Expected behavior:

- Agent calls **list\_directory** first
- Agent then calls **read\_file** for notes.txt
- Agent presents both results together

#### What to notice:

- Agent orchestrates multiple MCP tool calls
- Same patterns you learned in Module 1 apply to MCP tools



# Key takeaways

## What MCP is and why it matters:

- **Open standard** for connecting AI agents to external tool servers
- **Universal interoperability:** Works across AI frameworks (ADK, Claude, GPT)
- **Industry adoption:** Anthropic, OpenAI, Google, and The Linux Foundation
- **Ecosystem access:** Hundreds of pre-built tools for common tasks

## Connection types:

- **StdioConnectionParams** - Local servers via subprocess (development)
- **SseConnectionParams** - Remote servers via HTTP (production)

## Using MCP in ADK:

- **McpToolset** connects your agent to MCP servers
- **Tools are discovered automatically** - no manual registration
- **Use like any other tool** - same patterns from Module 1 apply

## Security with tool filtering:

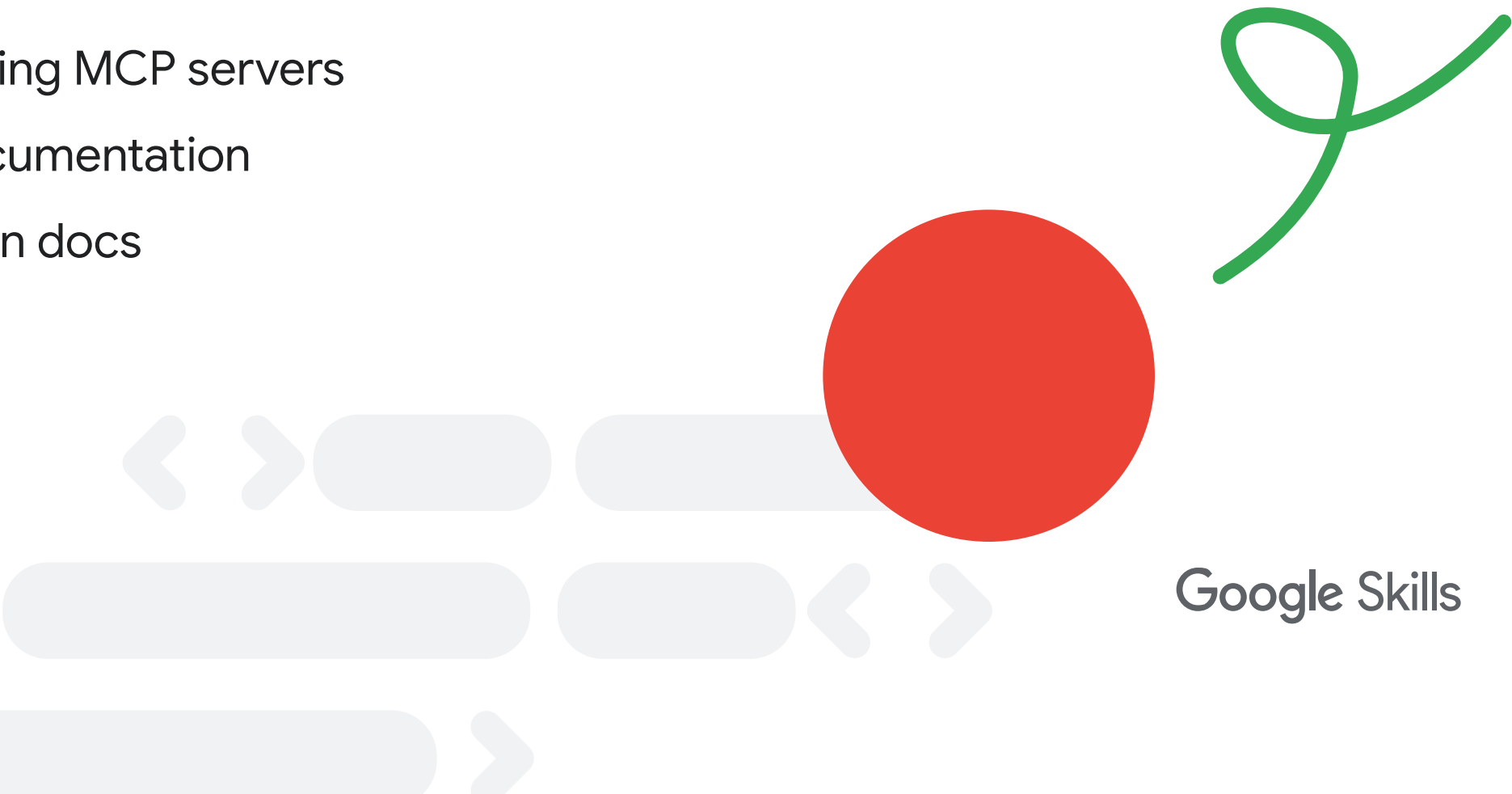
- Use **tool\_filter** to expose only needed tools
- **Limit to read-only** when possible for safety
- **Fewer tools = clearer decisions** for your agent

## When to use each tool type:

| Scenario                                         | Approach              | Why                         |
|--------------------------------------------------|-----------------------|-----------------------------|
| Pre-existing capability (files, databases, APIs) | MCP tools             | Community maintains it      |
| Google capabilities (search, code execution)     | Built-in tools        | ADK team maintains it       |
| Your business-specific logic                     | Custom function tools | Only you know your business |

## Key resources:

- **MCP Server Registry:** Find existing MCP servers
- **Official MCP docs:** Protocol documentation
- **ADK MCP guide:** ADK integration docs



# Beyond this module: Advanced MCP capabilities

This module covered the fundamentals of using existing MCP servers. MCP and ADK offer additional capabilities for more advanced use cases.

What else you can do with MCP in ADK:

| Capability                      | Description                                                          | Documentation                                   |
|---------------------------------|----------------------------------------------------------------------|-------------------------------------------------|
| Create your own MCP server      | Wrap your custom tools in an MCP server to share with others         | <a href="#">ADK docs - Building MCP servers</a> |
| Expose ADK tools via MCP        | Make your ADK tools accessible to any MCP client (Claude, GPT, etc.) | <a href="#">ADK docs - MCP tools</a>            |
| Connect to multiple MCP servers | Use tools from several MCP servers in a single agent                 | <a href="#">ADK docs - MCP tools</a>            |
| Deploy MCP servers to Cloud Run | Host your MCP servers for production use                             | <a href="#">ADK docs - MCP tools</a>            |

## The second integration pattern (advanced):

Earlier, we mentioned ADK supports two MCP integration patterns. The second pattern—**exposing ADK tools via an MCP server**—allows you to:

- 1 Wrap your existing ADK tools in an MCP server
- 2 Make those tools available to any MCP-compatible client
- 3 Share your tools with the broader AI ecosystem

This is valuable when you want other AI frameworks (Claude, GPT, etc.) to use tools you’ve built in ADK.

